Intermediate Code Generation

Translate from abstract-syntax trees to intermediate codes. Generating a low-level intermediate representation with two properties.

1- It should be easy to produce
2- It should be easy to translate into the target machine One of the popular

The benefits of using machine independent intermediate code are:

1. If a compiler translates the source language to its target machine language without having the ability to generate intermediate code, so for each new machine a full native compiler is required.
2. The intermediate code eliminates the need for a complete new compiler for every single machine by keeping the parsing part the same for all compilers.
3. The second part of the compiler, the synthesis, is modified depending on the target machine.
4. It becomes easier to apply source code changes to improve code performance by applying code optimization techniques on intermediate code.

If we generate machine code directly from source code then for n target machine we will have n optimizers and n code generators but if we will have a machine-independent intermediate code, we will have only one optimizer. Intermediate code can be either language specific (e.g., Bytecode for Java) or language independent (three-address code).

The following are commonly used intermediate code representation:

1- Postfix Notation –

The ordinary (infix) way of writing the sum of a and b is with operator in the middle: a + b

The postfix notation for the same expression places the operator at the right end as ab +. In general, if e1 and e2 are any postfix expressions, and + is any binary operator, the result of applying + to the values denoted by e1 and e2 is postfix notation by e1e2 +. No parentheses are needed in

postfix notation because the position and arity (number of arguments) of the operators permit only one way to decode a postfix expression. In postfix notation the operator follows the operand.

Example –
The postfix representation of the expression (a – b) * (c + d) + (a – b) is:
ab – cd + *ab -+.

2- Three-Address Code –

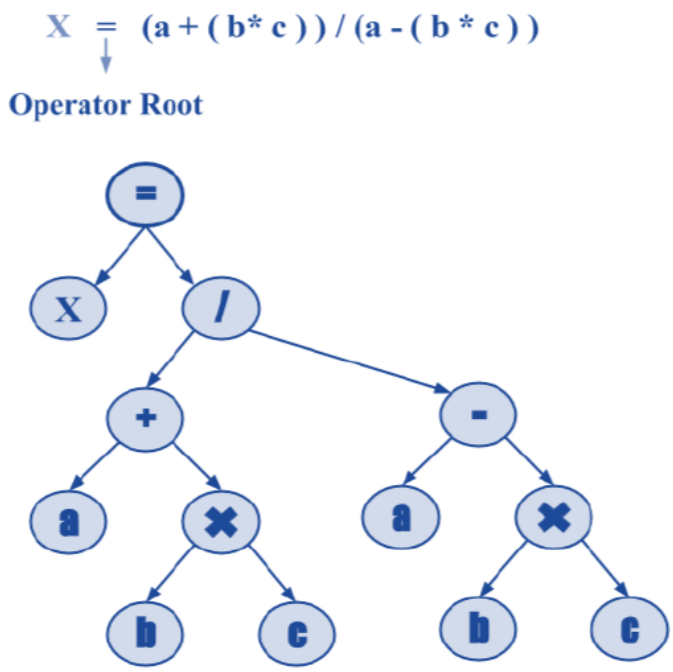
A statement involving no more than three references (two for operands and one for result) is known as three address statement. A sequence of three address statements is known as three address code. Three address statement is of the form
x = y op z where x, y, z will have address (memory location).
Sometimes a statement might contain less than three references but it is still called three address statement.

Example – The three address code for the expression a + b * c + d :

T 1 = b * c
T 2 = a + T 1
T 3 = T 2 + d
T 1 , T 2 , T 3 are temporary variables.

3- Syntax Tree –
Syntax tree is nothing more than condensed form of a parse tree. The operator and keyword nodes of the parse tree are moved to their parents and a chain of single productions is replaced by single link in syntax tree the internal nodes are operators and child nodes are operands. To form syntax tree put parentheses in the expression, this way it's easy to recognize which operand should come first.

Example – x = (a + b * c) / (a – b * c)



The three-address intermediate code .

Each statement contains at most 3 operands; in addition to
“:=”, i.e., assignment, at most one operator.

some of the basic operations which in the source program, to change in the Assembly language:

operations	**H.L.L**	**Assembly language**
Math. OP	+ , - , * , /	Add, sub, mult, div
Boolean. OP	&, | , ~	And, or, not
Assignment	=	mov
Jump	goto	JP, JN, JC
conditional	If, then	CMP
Loop instruction	For, do, repeat	These most have and I.C.G before
	until, while do	change it to assembly language.

The operation which change H.L.L to assembly language, is called the intermediate code generation and there is the division operation come with it, which mean every statement have a single operation.

Ex (1):

$$X = A + \underbrace{\underbrace{\underbrace{\underbrace{B * C}_{T1} / D}_{T2} - \underbrace{Y * N}_{T3}}_{T4}}_{T5}$$

T1 = B * C
T2 = T1 / D
T3 = Y * N
T4 = A + T2
T5 = T4 - T3

Ex (2):

$$Y = \underbrace{\underbrace{\underbrace{\cos (\underbrace{A * B}_{T1})}_{T5} + \underbrace{C / N}_{T4}}_{T6} - \underbrace{Y * P}_{T3}}_{T2}$$

T1 = A * B
T2 = cos T1 T3 = Y * P T4 = C / N T5 = T2 + T4 T6 = T5 – T3

Mathematical operation

There are two kinds of operation, which are deals with mathematical operation, such as the parsing for these operations:

1. Triple form

Ex: X = A + B * C / (- N)

	OP	**Arg1**	**Arg2**
(0)	*	B	C
(1)	-	N	__
(2)	/	(0)	(1)
(3)	+	A	(2)
	=	X	(3)

Ex: Y = A + C * X / B [i]

	OP	**Arg1**	**Arg2**
(0)	*	C	X
(1)	=[ ]	B	i
(2)	/	(0)	(1)
(3)	+	A	(2)
	=	Y	(3)

Ex: X[i] = N * C / Y[i]

	OP	**Arg1**	**Arg2**
(0)	*	N	C
(1)	=[ ]	Y	i
(2)	/	(0)	(1)
(3)	[ ]=	X	i
	=	(3)	(2)

Ex: X = A + B * (c / d) - y

	OP	**Arg1**	**Arg2**
(0)	/	c	d
(1)	*	B	(0)
(2)	+	A	(1)
(3)	-	(2)	y
	=	X	(3)

Ex: A = C * X [i,j]

	OP	**Arg1**	**Arg2**
(0)	=[ ]	X	P
(1)	*	C	(0)
	=	A	(1)

2. Quadruple form Ex: X = A

* C / N + P

OP	**Arg1**	**Arg2**	**Result**
*	A	C	t1
/	t1	N	t2
+	t2	P	t3
=	t3		X

Ex: A = N[i] * C / N

OP	**Arg1**	**Arg2**	**Result**
=[ ]	N	i	t1
*	t1	C	t2
/	t2	N	t3
=	t3		A

Ex: A = C * y / X[i,j]

OP	**Arg1**	**Arg2**	**Result**
*	C	y	t1
=[ ]	X	P	t2
/	t1	t2	t3
=	t3		A

Ex: X = A + B * (c / d) - y

OP	Arg1	Arg2	Result
/	c	d	t1
*	B	t1	t2
+	A	t2	t3
-	t3	y	t4
=	t4		X

Ex: X[i] = a * c + y[i] – n[j] / V

OP	Arg1	Arg2	Result
*	a	c	t1
=[ ]	n	j	t2
/	t2	V	t3
=[ ]	y	i	t4
+	t1	t4	t5
-	t5	t3	t6
[ ]=	X	i	t7
=	t6		t7

Code Optimization

Compilers should produce target code that is as good as can be written by hand. The code produced by straightforward compiling algorithms can often be made to run faster or take less space, or both. This improvement is achieved by program transformations that are traditionally called Optimizations.

Function- Preserving Transformations

There are several ways in which a compiler can improve a program without changing the function it computes. Common sub-expression elimination, copy propagation, dead- code elimination, and constant folding are common examples of such function- preserving transformations.

1. Common sub-expressions

Ex1: X = A + C * N – M
Y = B + C * N * e

Sol: Q = C * N
X = A + Q – M
Y = B + Q * e

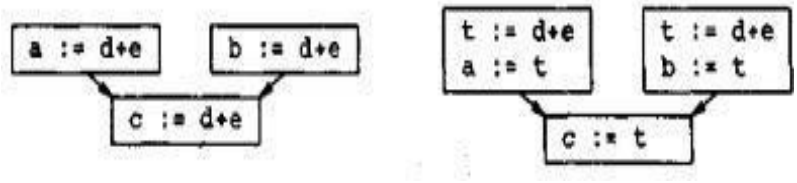
Ex2:

Before optimize	after optimize
t6 = 4 * i	t6 = 4 * i
X = a [t6]	X = a [t6]
t7 = 4 * i	t8 = 4 * j
t8 = 4 * j	t9 = a [t8]
t9 = a [t8]	a [t6] = t9
a [t7] = t9	a[t8] = X
t10 = 4 * j	
a[t10] = X	

<u>Note:</u> The value of the variable which are optimize will not be change.

2. Copy Propagation

Ex:



When the common sub-expression in c = d + e is eliminated in the previous section, the algorithm uses a new variable t to hold the value of d + e. Since control may reach c = d + e either after the assignment to a or after the assignment to b, it would be incorrect to replace c = d + e by either c=a or by c=b

🔷 Copy Propagation (in compiler optimization)

Copy Propagation is an optimization technique where the compiler replaces a variable that is simply a **copy of another variable** with the original variable itself.

If you have:

```
x = y
```

Then later:

```
z = x + 5
```

👉 Since `x` is just a copy of `y`, we can replace `x` with `y`:

```
z = y + 5
```

Example 1

Before Copy Propagation:

```
a = b
c = a + d
```

After Copy Propagation:

```
c = b + d
```

👉 The assignment `a = b` may become unnecessary and can be removed later.

Example 2 (with TAC)

Before:

```
t1 = A
t2 = t1 + B
```

After:

```
t2 = A + B
```

Example 3

```
t1 = A * B
t2 = t1
t3 = t2 + C
```

After Copy Propagation:

```
t1 = A * B
t3 = t1 + C
```

---

3. Dead- Code Elimination

A variable is live at a point in a program if its value can be used subsequently; otherwise, it is dead at that point. A related idea is dead or useless code, statements that compute values that never get used.

Ex1: X=3
If X > 4 Then
.
.
end

*This condition will not do, so we must use the optimization.

Ex2: A= false
If A Then Begin
.
.
end

*This condition also will not do

4. Loop Optimization

The running time of a program may be improved if we decrease the number of instructions in an inner loop, even if we increase the amount of code outside the loop. Three techniques are important for loop optimization: Code Motion, which moves code outside a loop; Induction Variable elimination; and, reduction in strength.

Code Motion

An important modification that decreases the amount of code in a loop is code motion. This transformation takes an expression that yields the same result independent of the number of times a loop is executed and places the expression before the loop.

Ex1: While (I <= limit-2)

Sol: t = limit-2
While (I <= t)

Ex2: While X<A + C*y do Begin
.
.
end

Sol: P = C * y
While X<A + P do Begin
.
.
end

Induction Variable

Ex: X = 2 * y + 2 * h + 4
Sol: X = 2 * (y + h) +4

<u>Note:</u> use the algebra method to optimize the mathematical expression.

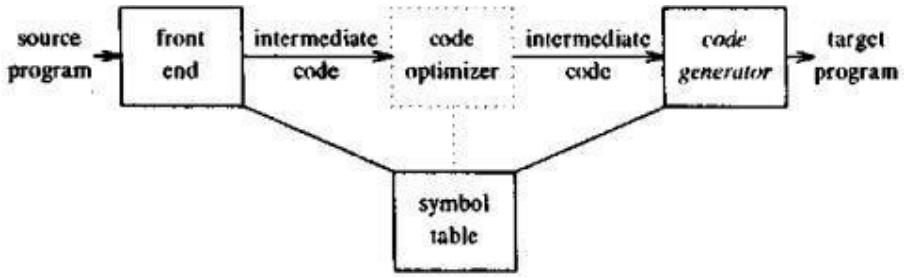
Reduction in strength

Reduction in strength, which replaces an expensive operation by a cheaper one, such as a multiplication by an addition.

Ex: t4 = 4*j - 4

Code Generation

The final phase in compiler is the code generator. It takes as input an intermediate representation of the source program and produces as output an equivalent target program, as indicated in Fig



Position of code generator

Code generation takes a linear sequence of 3-address intermediate code instructions, and translates each instruction into one or more instructions. The big issues in code generation are

1. Instruction selection
2. Register allocation and assignment

Instruction selection: for each type of three-address statement, we can design a code selection that outlines the target code to be generated for that construct.

Register allocation and assignment

The efficient utilization of registers involving operands is particularly important in generating good code. The use of registers is often subdivided into two sub-problems:

a. **Register allocation:** selecting the set of variables that will reside in registers at each point in the program

b. **Register assignment:** selecting a specific register in which a variable resides, the goal of these operations is generally to minimize the total number of memory accesses required by the program.